Tugas Kecil 1 IF2211

Strategi Algoritma

Penyelesaian Permainan Queens Linkedin

Disusun oleh:

Leonard Arif Sutiono / 18223120

PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

2025

Daftar Isi

Daftar Isi 2
1. Latar Belakang 3
2. Pembahasan 3
2.1. Penjelasan Algoritma Brute Force (Exhaustive Search) 3
2.1.1. Deskripsi Umum Algoritma 3
2.1.2. Representasi Data 3
2.1.3. Langkah-langkah Algoritma 4
2.1.4. Kompleksitas Algoritma 9
2.1.5. Karakteristik Algoritma 9
2.2. Source Code 10
2.2.1. File main.py 10
2.2.2. File QueenManager.py 10
2.2.3. File Board.py 11
2.2.4. File gui.py 18
2.3. Uji Coba 29
2.3.1 Test Case 1 (Kondisi Normal) 29
2.3.2 Test Case 2 (Kondisi Normal) 30
2.3.3 Test Case 3 (No Solution) 31
2.3.4 Test Case 4 (Kondisi Normal) 32
2.3.5 Test Case 5 (Input Invalid) 33
2.4. Link Repository Github 34
2.5. Surat Pernyataan 34
3. Lampiran 35

1. Latar Belakang

Queens adalah permainan logika yang mengharuskan pemain menempatkan bidak queen pada papan persegi berwarna. Permainan ini memiliki aturan bahwa setiap queen harus ditempatkan sedemikian rupa sehingga terdapat tepat satu queen pada setiap baris, kolom, dan daerah warna. Selain itu, tidak boleh ada queen yang bersebelahan satu sama lain, termasuk secara diagonal sebelahnya. Permasalahan ini merupakan variasi dari klasik N-Queens problem yang diperluas dengan constraint daerah warna, sehingga menciptakan ruang pencarian solusi yang sangat luas dan kompleks.

Algoritma brute force merupakan pendekatan fundamental dalam menyelesaikan permasalahan pencarian solusi. Algoritma ini bekerja dengan cara generate-and-test, yaitu membangkitkan semua kemungkinan kombinasi penempatan queen pada setiap daerah warna, kemudian menguji apakah kombinasi tersebut memenuhi seluruh constraint yang ada. Meskipun algoritma brute force memiliki kompleksitas waktu yang tinggi karena harus mengeksplorasi seluruh ruang pencarian, algoritma ini menjamin ditemukannya solusi apabila solusi tersebut ada. Dalam konteks pembelajaran, brute force memberikan pemahaman mendasar tentang exhaustive search dan menjadi baseline untuk membandingkan efisiensi algoritma-algoritma yang lebih kompleks.

Tugas kecil ini bertujuan untuk mengimplementasikan algoritma brute force murni tanpa heuristik dalam menyelesaikan permainan Queens. Program yang dikembangkan akan mampu membaca konfigurasi papan dari file, melakukan pencarian solusi secara sistematis dengan mencoba semua kombinasi penempatan queen, memvisualisasikan proses pencarian secara real-time, serta menampilkan hasil akhir serta jumlah iterasi yang dilakukan dan waktu eksekusi yang diperlukan.

2. Pembahasan

2.1. Penjelasan Algoritma Brute Force (Exhaustive Search)

2.1.1. Deskripsi Umum Algoritma

Algoritma brute force yang diimplementasikan dalam program ini menggunakan pendekatan generate-and-test dengan strategi enumerasi kombinatorial. Algoritma ini mencoba semua kemungkinan kombinasi penempatan queen pada setiap daerah warna yang ada di papan, kemudian memvalidasi setiap kombinasi tersebut terhadap constraint permainan.

2.1.2. Representasi Data

Sebelum menjelaskan langkah-langkah algoritma, perlu dipahami terlebih dahulu bagaimana data direpresentasikan:

- Struktur Papan: Papan permainan disimpan dalam array 2D (self.display) dimana setiap sel berisi karakter yang merepresentasikan daerah warna.
- Struktur Warna: Informasi daerah warna disimpan dalam dictionary (self.colors) yang memetakan setiap karakter warna ke list posisi koordinat (x, y) yang termasuk dalam daerah tersebut. Contoh:
- Array Indeks: Array indexPerColor digunakan untuk melacak posisi queen yang dipilih pada setiap daerah warna. Misalnya, jika ada 9 daerah warna (A sampai I), maka array ini memiliki 9 elemen, dimana setiap elemen merepresentasikan indeks posisi yang dipilih pada daerah warna tersebut.

2.1.3. Langkah-langkah Algoritma

Langkah 1: Inisialisasi

Algoritma dimulai dengan membaca dan memproses papan dari file input melalui fungsi loadFromFile(). Proses ini mengidentifikasi:

- Dimensi papan (jumlah baris dan kolom)
- Daerah-daerah warna yang ada
- Posisi-posisi sel yang termasuk dalam setiap daerah warna

Papan juga akan divalidasi dengan fungsi checkBoard() agar panjang dan lebar papan sesuai dan banyak warna juga sama jumlahnya dengan panjang dan lebar papan.

```
def checkBoard(self):
    if self.row != self.col:
        return False
    if len(self.colors) != self.row or len(self.colors) != self.col:
        return False
    return True
```

Setelah itu, algoritma membuat list dari semua daerah warna (colorsList) dan array indexPerColor yang diinisialisasi dengan nilai 0 untuk semua elemen. Array ini berfungsi sebagai "counter" yang akan di-increment untuk mencoba semua kombinasi.

```
colorsList = list(self.colors.values())
k = len(colorsList)
if k == 0:
    return False
indexPerColor = [0] * k
```

Sebelum melanjutkan tahapan selanjutnya, dibuat juga salinan dari papan kosongnya agar bisa melakukan reset jika solusi masih belum valid.

```
originalDisplay = [row[:] for row in self.display]
```

Langkah 2: Loop Utama - Generate Kombinasi

Algoritma memasuki loop while True yang akan terus berjalan hingga solusi ditemukan atau semua kombinasi telah dicoba. Dalam setiap iterasi loop:

a. Reset papan

```
self.display = [row[:] for row in originalDisplay]
queenManager.__init__(self)
```

Papan dikembalikan ke kondisi awal (tanpa queen) dan QueenManager direset untuk memulai konfigurasi baru.

b. Penempatan Queen Berdasarkan Kombinasi Saat Ini

```
self.placeQueens(queenManager, indexPerColor)
```

Berikut rincian fungsi placeQueens():

```
def placeQueens(self, queenManager, indexPerColor):
    i = 0
    for position in self.colors.values():
        x, y = position[indexPerColor[i]]
        queenManager.place(x,y)
        i += 1
```

Fungsi ini menempatkan queen pada setiap daerah warna berdasarkan indeks yang disimpan dalam indexPerColor. Misalnya:

- Untuk daerah warna pertama (A), pilih posisi ke-indexPerColor[0]
- Untuk daerah warna kedua (B), pilih posisi ke-indexPerColor[1]
- Dan seterusnya untuk semua daerah warna

Langkah 3: Validasi Kombinasi

Setelah queen ditempatkan, algoritma melakukan validasi melalui fungsi validChecker() untuk memastikan kombinasi tersebut memenuhi semua constraint.

```
def validChecker(self):
    queenX = set()
    queenY = set()
    for i in range(self.row):
        for j in range(self.col):
            if self.display[i][j] == '#':
                if i in queenY:
                    return False

                if j in queenX:
                    return False

                if not self.checkDiagonal(j,i):
                    return False

                queenX.add(j)
                queenY.add(i)

    return True
```

a. Cek bari dan kolom.

```
for i in range(self.row):
    for j in range(self.col):
        if self.display[i][j] == '#':
            if i in queenY:
                return False

            if j in queenX:
                return False
```

Algoritma memeriksa bahwa tidak ada dua queen pada baris atau kolom yang sama dengan menggunakan struktur data set untuk efisiensi.

b. Cek diagonal

```
if not self.checkDiagonal(j,i):
    return False
```

Berikut isi dari fungsi checkDiagonal:

```
def checkDiagonal(self, x,y):
    max_x = self.col
    max_y = self.row
    directions = [(-1, 1), (-1, -1), (1, 1), (1, -1)]

    for dy, dx in directions:
        ny = y + dy
        nx = x + dx

        if 0 <= nx < max_x and 0 <= ny < max_y:
            if self.display[ny][nx] == '#':
                return False

    return True
```

Fungsi checkDiagonal() memeriksa 4 arah diagonal (kiri-atas, kanan-atas, kiri-bawah, kanan-bawah) untuk memastikan tidak ada queen yang bersebelahan secara diagonal

c. Constraint Daerah Warna
Constraint daerah warna secara implisit sudah terpenuhi karena algoritma menempatkan tepat satu queen pada setiap daerah warna melalui mekanisme indexPerColor.

Langkah 4: Evaluasi Hasil Validasi

```
if self.validChecker():
    print("found after", tried, "combinations")
    return True, self.display
```

Jika validasi mengembalikan True, berarti solusi valid telah ditemukan. Algoritma mengembalikan hasil dan berhenti.

Langkah 5: Generate Kombinasi Berikutnya

Jika kombinasi saat ini tidak valid, algoritma perlu mencoba kombinasi berikutnya dengan memanggil fungsi moveQueen():

```
if not self.moveQueen(indexPerColor):
    print("no solution after", tried, "combinations")
    return False, originalDisplay
```

Fungsi moveQueen() bekerja seperti counter dengan basis berbeda-beda untuk setiap digit (karena setiap daerah warna bisa memiliki jumlah posisi yang berbeda):

```python
def moveQueen(self, arr):
    colorsList = list(self.colors.values())
    maxIndexPerColor = [len(colorsList[i]) for i in range(len(colorsList))]

    i = len(arr) -1
    while i >= 0:
        arr[i] += 1
        if arr[i] < maxIndexPerColor[i]:
            return True

        else:
            arr[i] = 0
            i -= 1

    return False
```

Misalkan ada 3 daerah warna dengan jumlah posisi [3, 2, 4], maka kombinasi akan di generate dalam urutan:

```
[0,0,0] → [0,0,1] → [0,0,2] → [0,0,3] → [0,1,0] → [0,1,1] → [0,1,2] →
[0,1,3] → [1,0,0] → [1,0,1] → ... → [2,1,3] (kombinasi terakhir)
```

Jika moveQueen() mengembalikan False, berarti semua kombinasi telah dicoba dan tidak ada solusi yang valid.

2.1.4. Kompleksitas Algoritma

Kompleksitas Waktu: $O(P_1 \times P_2 \times ... \times P_k \times V)$

Dimana:

- k = jumlah daerah warna
- P_i = jumlah posisi pada daerah warna ke-i
- V = kompleksitas validasi = $O(N^2)$ untuk papan N×N

Dalam kasus terburuk (tidak ada solusi), algoritma harus mencoba semua kombinasi penempatan queen, yaitu hasil perkalian jumlah posisi dari setiap daerah warna. Kompleksitas Ruang: $O(N^2)$, Untuk menyimpan papan, informasi warna, dan array indeks.

2.1.5. Karakteristik Algoritma

Kelebihan:

- Completeness: Algoritma dijamin menemukan solusi jika solusi tersebut ada

- Simplicity: Implementasi straightforward dan mudah dipahami
- No Heuristic: Murni brute force tanpa asumsi domain-specific

Kekurangan:

- Efisiensi: Kompleksitas eksponensial membuat algoritma lambat untuk kasus besar
- Redundansi: Mencoba banyak kombinasi yang jelas tidak valid (misalnya dua queen pada kolom yang sama)

2.2. Source Code

2.2.1. File main.py

```
from gui import BoardGUI

if __name__ == "__main__":
    app = BoardGUI()
    app.mainloop()
```

File main.py merupakan entry point atau titik masuk utama dari program. File ini berisi kode yang sangat sederhana dengan hanya beberapa baris yang berfungsi untuk menginisialisasi dan menjalankan aplikasi GUI. Di dalam blok kondisional if __name__ == "__main__", program menginstansiasi objek BoardGUI yang merupakan class GUI utama, kemudian memanggil method mainloop() untuk memulai event loop dari aplikasi Tkinter. Event loop ini akan terus berjalan dan menunggu interaksi dari pengguna seperti klik tombol atau input lainnya hingga aplikasi ditutup. Pemisahan file main ini mengikuti best practice dalam pengembangan aplikasi Python, dimana entry point dipisahkan dari logika bisnis utama sehingga memudahkan testing dan modularitas.

2.2.2. File QueenManager.py

```
class QueenManager:
    def __init__(self, board):
        self.board = board
        self.queensX = []
        self.queensY = []
        self.occupied_colors = []
        self.total = 0

    def place(self, x, y):
```

```
        b = self.board
        self.queensX.append(x)
        self.queensY.append(y)
        self.occupied_colors.append(b.display[y][x])
        self.total += 1
        b.display[y][x] = '#'

    def removeLatest(self):
        b = self.board
        x = self.queensX.pop()
        y = self.queensY.pop()
        color = self.occupied_colors.pop()
        self.total -= 1
        b.display[y][x] = color
        return x, y
```

File QueenManager.py mengimplementasikan class QueenManager yang bertanggung jawab untuk mengelola state dan operasi yang berkaitan dengan penempatan queen pada papan permainan. Class ini menyimpan informasi penting seperti koordinat x dan y dari semua queen yang telah ditempatkan dalam list queensX dan queensY, serta menyimpan informasi warna dari sel-sel yang ditempati queen dalam list occupied_colors. Method place(x, y) digunakan untuk menempatkan queen pada posisi tertentu dengan cara menambahkan koordinat ke dalam list, menyimpan warna sel tersebut, dan mengganti karakter pada papan dengan simbol '#' yang merepresentasikan queen. Sebaliknya, method removeLatest() digunakan untuk membatalkan penempatan queen terakhir dengan melakukan operasi pop pada semua list dan mengembalikan karakter warna asli ke papan. Class ini menggunakan konsep encapsulation dengan menyimpan referensi ke objek board sehingga dapat melakukan modifikasi langsung pada state papan, dan sangat penting dalam implementasi algoritma backtracking dimana operasi place dan remove dilakukan berulang kali.

2.2.3. File Board.py

```
import time
import os


class Board:
```

```python
    def __init__(self):
        self.row = 0
        self.col = 0
        self.colors = {}
        self.display = []


    def loadFromFile(self, fileName):
        self.row = 0
        self.col = 0
        self.colors = {}
        self.display = []

        with open(fileName, 'r') as file:
            lines = file.readlines()

        expected_len = None

        for y, line in enumerate(lines):
            row = list(line.strip('\n'))

            if expected_len is None:
                expected_len = len(row)
            elif len(row) != expected_len:
                raise ValueError("Setiap baris pada papan harus memiliki
panjang yang sama.")

            self.display.append(row)
            for x, ch in enumerate(row):
                if ch not in self.colors:
                    self.colors[ch] = []
                self.colors[ch].append((x,y))

        self.row = len(self.display)
        self.col = len(self.display[0])

    def checkDiagonal(self, x,y):
        max_x = self.col
```

```python
        max_y = self.row
        directions = [(-1, 1), (-1, -1), (1, 1), (1, -1)]

        for dy, dx in directions:
            ny = y + dy
            nx = x + dx

            if 0 <= nx < max_x and 0 <= ny < max_y:
                if self.display[ny][nx] == '#':
                    return False

        return True

    def placeQueens(self, queenManager, indexPerColor):
        i = 0
        for position in self.colors.values():
            x, y = position[indexPerColor[i]]
            queenManager.place(x,y)
            i += 1

    def validChecker(self):
        queenX = set()
        queenY = set()
        for i in range(self.row):
            for j in range(self.col):
                if self.display[i][j] == '#':
                    if i in queenY:
                        return False

                    if j in queenX:
                        return False

                    if not self.checkDiagonal(j,i):
                        return False

                    queenX.add(j)
                    queenY.add(i)
```

```
        return True

    def moveQueen(self, arr):
        colorsList = list(self.colors.values())
        maxIndexPerColor = [len(colorsList[i]) for i in range(len(colorsList))]

        i = len(arr) -1
        while i >= 0:
            arr[i] += 1
            if arr[i] < maxIndexPerColor[i]:
                return True

            else:
                arr[i] = 0
                i -= 1

        return False

    def exhaustiveSearch(self, queenManager, fileName, progressCallback=None):
        colorsList = list(self.colors.values())
        k = len(colorsList)
        if k == 0:
            return False
        indexPerColor = [0] * k

        originalDisplay = [row[:] for row in self.display]
        tried = 0
        lastUpdate = time.time()

        while True:
            self.display = [row[:] for row in originalDisplay]
            queenManager.__init__(self)

            self.placeQueens(queenManager, indexPerColor)

            tried += 1
            if tried % 1000 == 0:
                print("tried", tried, "combinations:", indexPerColor)
```

```
            if progressCallback is not None:
                now = time.time()
                if now - lastUpdate >= 1.0:
                    positions = list(zip(queenManager.queensX,
queenManager.queensY))
                    try:
                        progressCallback(positions, tried)
                    except Exception:
                        pass
                    lastUpdate = now

            if self.validChecker():
                print("found after", tried, "combinations")
                return True, self.display

            if not self.moveQueen(indexPerColor):
                print("no solution after", tried, "combinations")
                return False, originalDisplay

    def backtrack(self, row, queenManager):
        if row == self.row or len(queenManager.occupied_colors) ==
len(self.colors):
            return True

        for col in range(self.col):
            if (self.display[row][col] not in queenManager.occupied_colors and
                col not in queenManager.queensX and
                row not in queenManager.queensY and
                self.checkDiagonal(col,row)):

                queenManager.place(col, row)

                if self.backtrack(row + 1, queenManager):
                    return True

                queenManager.removeLatest()
```

```
        return False

    def checkBoard(self):
        if self.row != self.col:
            return False
        if len(self.colors) != self.row or len(self.colors) != self.col:
            return False
        return True

    def solveExhaustive(self, queenManager, fileName):
        self.loadFromFile(fileName)

        if not self.checkBoard():
            print("Board tidak valid: papan harus NxN dan memiliki N warna
unik.")
            return

        queenManager.__init__(self)

        start_time = time.time()
        ok, results = self.exhaustiveSearch(queenManager, fileName)
        end_time = time.time()
        elapsed = (end_time - start_time)*1000

        print(f"Waktu pencarian (exhaustive): {elapsed:.4f} ms")

        if not ok:
            print("Tidak ada solusi yang ditemukan (exhaustive).")
            return None

        for row in results:
            print("".join(row))

        self.writeSolutionToFile(fileName, elapsed, results, "exhaustive")

        return self.display

    def solveBacktrack(self, queenManager, fileName):
```

```
        self.loadFromFile(fileName)

        if not self.checkBoard():
            print("Board tidak valid: papan harus NxN dan memiliki N warna
unik.")
            return

        queenManager.__init__(self)

        start_time = time.time()
        found = self.backtrack(0,queenManager)
        end_time = time.time()
        elapsed = (end_time - start_time)*1000

        print(f"Waktu pencarian (backtrack): {elapsed:.4f} ms")

        if not found:
            print("Tidak ada solusi yang ditemukan (backtrack).")
            return None

        for row in self.display:
            print("".join(row))

        self.writeSolutionToFile(fileName, elapsed, self.display, "backtrack")

        return self.display

    def writeSolutionToFile(self, fileName, elapsed, board_state, method_name):
        base = os.path.basename(fileName)
        name_no_ext, _ = os.path.splitext(base)

        dir_name = os.path.dirname(fileName)
        if not dir_name:
            dir_name = "test"

        os.makedirs(dir_name, exist_ok=True)

        output_path = os.path.join(dir_name, f"{name_no_ext}_solution.txt")
```

```
        try:
            with open(output_path, 'w') as f:
                f.write(f"Metode: {method_name}\n")
                f.write(f"Waktu pencarian: {elapsed:.4f} ms\n")
                for row in board_state:
                    f.write("".join(row) + "\n")
            print(f"Solusi disimpan di: {output_path}")
        except Exception as e:
            print(f"Gagal menulis file solusi: {e}")
```

File Board.py merupakan inti dari program yang mengimplementasikan class Board berisi seluruh logika bisnis dan algoritma penyelesaian permainan Queens. Class ini memiliki beberapa atribut penting seperti row dan col untuk menyimpan dimensi papan, colors sebagai dictionary yang memetakan setiap karakter warna ke list koordinat sel yang memiliki warna tersebut, dan display sebagai array 2D yang merepresentasikan state papan saat ini. Method loadFromFile() bertanggung jawab membaca file input dan melakukan parsing untuk mengekstrak informasi papan dan daerah warna. Method validChecker() melakukan validasi komprehensif terhadap konfigurasi queen dengan mengecek constraint baris, kolom, dan diagonal, menggunakan struktur data set untuk efisiensi pengecekan duplikasi. Method checkDiagonal() secara spesifik memvalidasi constraint diagonal dengan mengecek empat arah diagonal dari posisi queen. Method placeQueens() menempatkan queen berdasarkan array indeks yang merepresentasikan kombinasi yang sedang dicoba. Method moveQueen() mengimplementasikan mekanisme counter untuk menggenerate kombinasi berikutnya dengan prinsip carry-over seperti pada sistem penghitungan dengan basis berbeda-beda. Method exhaustiveSearch() adalah implementasi algoritma brute force utama yang melakukan iterasi melalui semua kombinasi, dilengkapi dengan fitur progress callback untuk visualisasi real-time dan tracking jumlah kombinasi yang dicoba. Sebagai perbandingan, file ini juga mengimplementasikan method backtrack() yang merupakan algoritma alternatif dengan pendekatan rekursif yang lebih efisien. Method solveExhaustive() dan solveBacktrack() adalah wrapper function yang mengkoordinasikan proses pembacaan file, eksekusi algoritma, pengukuran waktu, dan penulisan hasil ke file output melalui method writeSolutionToFile().

2.2.4. File gui.py

```
import os
import time
import tkinter as tk
```

```python
from tkinter import filedialog, messagebox
import threading
from Board import Board
from QueenManager import QueenManager

try:
    from PIL import Image, ImageGrab
    PIL_AVAILABLE = True
except ImportError:
    PIL_AVAILABLE = False


CELL_SIZE = 40
PADDING = 20
ANIMATION_DELAY_MS = 300


class BoardGUI(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("Colored Queens")
        self.configure(padx=20, pady=20)

        self.board = Board()
        self.qm = QueenManager(self.board)
        self.filePath = None
        self.queenPositions = []
        self.currentAnimIndex = 0
        self.isSolving = False
        self.baseDisplay = []

        self.buildWidgets()

    def buildWidgets(self):
        topFrame = tk.Frame(self)
        topFrame.pack(side=tk.TOP, fill=tk.X, padx=10, pady=10)

        self.fileEntry = tk.Entry(topFrame, width=40)
```

```
        self.fileEntry.pack(side=tk.LEFT, padx=5)

        browseButton = tk.Button(topFrame, text="Browse...",
command=self.browseFile)
        browseButton.pack(side=tk.LEFT, padx=5)

        solveExhaustiveButton = tk.Button(topFrame, text="Solve Exhaustive",
                                          command=self.solveExhaustive)
        solveExhaustiveButton.pack(side=tk.LEFT, padx=5)

        solveBacktrackButton = tk.Button(topFrame, text="Solve Backtrack",
                                         command=self.solveBacktrack)
        solveBacktrackButton.pack(side=tk.LEFT, padx=5)

        clearButton = tk.Button(topFrame, text="Clear Board",
command=self.clearBoard)
        clearButton.pack(side=tk.LEFT, padx=5)

        saveButton = tk.Button(topFrame, text="Save Image",
command=self.saveImage)
        saveButton.pack(side=tk.LEFT, padx=5)

        self.canvas = tk.Canvas(self, bg="white")
        self.canvas.pack(side=tk.TOP, padx=10, pady=10)

    def browseFile(self):
        path = filedialog.askopenfilename(
            initialdir=".",
            title="Select board file",
            filetypes=(("Text files", "*.txt"), ("All files", "*.*")),
        )
        if path:
            self.filePath = path
            self.fileEntry.delete(0, tk.END)
            self.fileEntry.insert(0, path)
            try:
                self.board.loadFromFile(path)
                self.baseDisplay = [row[:] for row in self.board.display]
```

```
            self.resizeCanvas()
            self.drawBoard(backgroundOnly=True)
        except Exception as e:
            messagebox.showerror("Error", f"Failed to load file:\n{e}")

    def resizeCanvas(self):
        w = self.board.col * CELL_SIZE + 2 * PADDING
        h = self.board.row * CELL_SIZE + 2 * PADDING
        self.canvas.config(width=w, height=h)

    def drawBoard(self, queens=None, backgroundOnly=False):
        self.canvas.delete("all")

        if not self.board.display:
            return

        source = self.baseDisplay if self.baseDisplay else self.board.display

        color_map = {}
        palette = ["#f2f2f2", "#d9ead3", "#c9daf8", "#f4cccc",
                   "#fff2cc", "#d0e0e3", "#ead1dc", "#cfe2f3"]
        palette_idx = 0

        for y in range(self.board.row):
            for x in range(self.board.col):
                ch = source[y][x]

                if ch not in color_map:
                    color_map[ch] = palette[palette_idx % len(palette)]
                    palette_idx += 1

                x1 = PADDING + x * CELL_SIZE
                y1 = PADDING + y * CELL_SIZE
                x2 = x1 + CELL_SIZE
                y2 = y1 + CELL_SIZE

                self.canvas.create_rectangle(
                    x1, y1, x2, y2,
```

```
                fill=color_map[ch],
                outline="black"
            )
            self.canvas.create_text(
                (x1 + x2) // 2,
                (y1 + y2) // 2,
                text=ch,
                font=("Arial", 14, "bold")
            )

    if not backgroundOnly and queens:
        for qx, qy in queens:
            self.drawQueen(qx, qy)

def drawQueen(self, x, y):
    x1 = PADDING + x * CELL_SIZE + 4
    y1 = PADDING + y * CELL_SIZE + 4
    x2 = x1 + CELL_SIZE - 8
    y2 = y1 + CELL_SIZE - 8
    self.canvas.create_oval(x1, y1, x2, y2, fill="black")
    self.canvas.create_text(
        (x1 + x2) // 2,
        (y1 + y2) // 2,
        text="Q",
        fill="white",
        font=("Arial", 14, "bold")
    )

def collectQueenPositionsFromManager(self):
    return list(zip(self.qm.queensX, self.qm.queensY))

def startAnimation(self):
    self.drawBoard(backgroundOnly=True)
    self.currentAnimIndex = 0
    self.animateStep()

def animateStep(self):
    if self.currentAnimIndex >= len(self.queenPositions):
```

```
            return
        qx, qy = self.queenPositions[self.currentAnimIndex]
        self.drawQueen(qx, qy)
        self.currentAnimIndex += 1
        self.after(ANIMATION_DELAY_MS, self.animateStep)

    def solveExhaustive(self):
        if self.isSolving:
            return

        path = self.fileEntry.get().strip()
        if not path:
            messagebox.showwarning("Warning", "Please select a board file
first.")
            return

        self.isSolving = True

        t = threading.Thread(target=self.workerExhaustive, args=(path,),
daemon=True)
        t.start()

    def workerExhaustive(self, path):
        try:
            self.board.loadFromFile(path)
            # Validasi papan sebelum mulai exhaustive search
            if not self.board.checkBoard():
                raise ValueError("Board tidak valid: papan harus NxN dan
memiliki N warna unik.")

            self.baseDisplay = [row[:] for row in self.board.display]
            self.qm = QueenManager(self.board)

            def progressCallback(positions, tried):
                self.after(0, lambda p=positions, t=tried:
self.onExhaustiveProgress(p, t))

            start_time = time.time()
```

```
            ok, board_state = self.board.exhaustiveSearch(self.qm, path,
progressCallback)
            elapsed = (time.time() - start_time)*1000

            if ok:
                print(f"Waktu pencarian (exhaustive): {elapsed:.4f} ms")
                try:
                    self.board.writeSolutionToFile(path, elapsed, board_state,
"exhaustive")
                except Exception as e:
                    print(f"Gagal menyimpan solusi exhaustive: {e}")

            positions = self.collectQueenPositionsFromManager() if ok else []
        except Exception as e:
            # Ikat exception ke argumen default lambda agar aman di Python
3.11+
            self.after(0, lambda err=e: self.onExhaustiveDone(path, False, [],
err))
            return

        self.after(0, lambda: self.onExhaustiveDone(path, ok, positions, None))

    def onExhaustiveProgress(self, positions, tried):
        self.queenPositions = positions
        self.drawBoard(queens=self.queenPositions)

    def onExhaustiveDone(self, path, ok, positions, error):
        self.isSolving = False

        if error is not None:
            messagebox.showerror("Invalid Board", "Board harus NxN dan memiliki
N warna unik.")
            return

        if not ok:
            messagebox.showinfo("Info", "No solution found (exhaustive).")
            return
```

```
        self.queenPositions = positions

        try:
            self.board.loadFromFile(path)
            self.baseDisplay = [row[:] for row in self.board.display]
            self.resizeCanvas()
        except Exception as e:
            messagebox.showerror("Error", f"Failed to reload board:\n{e}")
            return
        self.drawBoard(queens=self.queenPositions)

    def solveBacktrack(self):
        path = self.fileEntry.get().strip()
        if not path:
            messagebox.showwarning("Warning", "Please select a board file
first.")
            return

        try:
            self.board.loadFromFile(path)
            if not self.board.checkBoard():
                messagebox.showerror("Invalid Board", "Board harus NxN dan
memiliki N warna unik.")
                return

            self.baseDisplay = [row[:] for row in self.board.display]
            self.resizeCanvas()
            self.qm = QueenManager(self.board)
            result = self.board.solveBacktrack(self.qm, path)

            # Jika tidak ada solusi, tampilkan pesan di GUI dan berhenti
            if result is None or not self.qm.queensX:
                messagebox.showinfo("Info", "No solution found (backtrack).")
                return

            self.queenPositions = self.collectQueenPositionsFromManager()

            self.board.loadFromFile(path)
```

```python
            self.baseDisplay = [row[:] for row in self.board.display]
            self.startAnimation()
        except Exception as e:
            messagebox.showerror("Error", f"Backtrack solve failed:\n{e}")

    def clearBoard(self):
        if not self.board.display:
            return
        try:
            if self.filePath:
                self.board.loadFromFile(self.filePath)
                self.baseDisplay = [row[:] for row in self.board.display]
        except Exception:
            pass
        self.queenPositions = []
        self.qm = QueenManager(self.board)
        self.drawBoard(backgroundOnly=True)

    def saveImage(self):
        if not self.board.display or not self.queenPositions:
            messagebox.showwarning("Warning", "No solution to save yet.")
            return

        baseDir = os.path.dirname(os.path.abspath(__file__))
        outputDir = os.path.join(baseDir, "..", "output")
        os.makedirs(outputDir, exist_ok=True)

        if not PIL_AVAILABLE:
            psPath = os.path.join(outputDir, "solution.ps")
            self.canvas.postscript(file=psPath)
            messagebox.showinfo(
                "Saved",
                f"PostScript saved to {psPath}\n"
                f"Install Pillow to save PNG automatically."
            )
            return

        try:
```

```python
            pngPath = os.path.join(outputDir, "solution.png")
            self.renderBoardToPNG(pngPath)
            messagebox.showinfo("Saved", f"Image saved to {pngPath}")

        except Exception as e:
            messagebox.showerror("Error", f"Failed to save PNG:\n{e}")


    def renderBoardToPNG(self, filepath):
        """Render board directly to PNG using PIL"""
        from PIL import Image, ImageDraw, ImageFont

        width = self.board.col * CELL_SIZE + 2 * PADDING
        height = self.board.row * CELL_SIZE + 2 * PADDING

        img = Image.new('RGB', (width, height), 'white')
        draw = ImageDraw.Draw(img)

        try:
            font = ImageFont.truetype("arial.ttf", 14)
            font_bold = ImageFont.truetype("arialbd.ttf", 14)
        except:
            font = ImageFont.load_default()
            font_bold = font

        source = self.baseDisplay if self.baseDisplay else self.board.display
        color_map = {}
        palette = ["#f2f2f2", "#d9ead3", "#c9daf8", "#f4cccc",
                "#fff2cc", "#d0e0e3", "#ead1dc", "#cfe2f3"]
        palette_idx = 0

        for y in range(self.board.row):
            for x in range(self.board.col):
                ch = source[y][x]

                if ch not in color_map:
                    color_map[ch] = palette[palette_idx % len(palette)]
                    palette_idx += 1
```

```
            x1 = PADDING + x * CELL_SIZE
            y1 = PADDING + y * CELL_SIZE
            x2 = x1 + CELL_SIZE
            y2 = y1 + CELL_SIZE

            draw.rectangle([x1, y1, x2, y2],
                           fill=color_map[ch],
                           outline='black',
                           width=1)

            bbox = draw.textbbox((0, 0), ch, font=font_bold)
            text_width = bbox[2] - bbox[0]
            text_height = bbox[3] - bbox[1]
            text_x = x1 + (CELL_SIZE - text_width) // 2
            text_y = y1 + (CELL_SIZE - text_height) // 2

            draw.text((text_x, text_y), ch, fill='black', font=font_bold)

    for qx, qy in self.queenPositions:
        x1 = PADDING + qx * CELL_SIZE + 4
        y1 = PADDING + qy * CELL_SIZE + 4
        x2 = x1 + CELL_SIZE - 8
        y2 = y1 + CELL_SIZE - 8

        draw.ellipse([x1, y1, x2, y2], fill='black', outline='black')

        bbox = draw.textbbox((0, 0), "Q", font=font_bold)
        text_width = bbox[2] - bbox[0]
        text_height = bbox[3] - bbox[1]
        text_x = x1 + (CELL_SIZE - 8 - text_width) // 2
        text_y = y1 + (CELL_SIZE - 8 - text_height) // 2

        draw.text((text_x, text_y), "Q", fill='white', font=font_bold)

    img.save(filepath, 'PNG')
```

File gui.py mengimplementasikan class BoardGUI yang merupakan graphical user interface (GUI) berbasis Tkinter untuk memvisualisasikan papan permainan dan proses pencarian solusi. Class ini mewarisi dari tk.Tk yang merupakan root window dari aplikasi Tkinter. Method buildWidgets() membangun komponen-komponen UI seperti text entry untuk menampilkan path file, tombol browse untuk memilih file input, tombol untuk menjalankan algoritma exhaustive search dan backtracking, tombol clear untuk mereset papan, tombol save untuk menyimpan gambar solusi, dan canvas untuk menggambar visualisasi papan. Method browseFile() membuka dialog pemilihan file dan memuat papan dari file yang dipilih. Method drawBoard() melakukan rendering papan pada canvas dengan menggambar persegi panjang berwarna untuk setiap sel dan menempatkan label karakter warna, menggunakan palette warna yang telah ditentukan untuk membedakan daerah warna secara visual. Method drawQueen() menggambar representasi visual queen sebagai lingkaran hitam dengan huruf 'Q' putih di tengahnya. Method solveExhaustive() menjalankan algoritma exhaustive search dalam thread terpisah melalui workerExhaustive() untuk mencegah GUI freeze selama proses pencarian yang bisa memakan waktu lama. Method onExhaustiveProgress() dipanggil secara periodik untuk update visualisasi selama proses pencarian berlangsung, memberikan feedback real-time kepada pengguna. Method solveBacktrack() menjalankan algoritma backtracking dan menganimasikan penempatan queen secara bertahap melalui method startAnimation() dan animateStep() untuk memberikan efek visual yang menarik. Method saveImage() memungkinkan user untuk menyimpan hasil solusi dalam format gambar PNG menggunakan library Pillow, dengan method helper renderBoardToPNG() yang melakukan rendering papan ke dalam objek image PIL kemudian menyimpannya ke file. GUI ini memberikan pengalaman interaktif yang user-friendly dengan visualisasi yang jelas, memudahkan pengguna untuk memahami proses kerja algoritma dan hasil yang dihasilkan.

2.3. Uji Coba

2.3.1 Test Case 1 (Kondisi Normal)

Input Test Case 1:

```
AAABBCCCD
ABBBBCECD
ABBBDCECD
AAABDCCCD
BBBBDDDDD
FGGGDDHDD
FGIGDDHDD
FGIGDDHDD
FGGGDDHHH
```

Output txt Test Case 1:

```
Metode: exhaustive
Waktu pencarian: 212585.8319 ms
AAABBCC#D
ABBB#CECD
```

```
ABBBDC#CD
A#ABDCCCD
BBBBD#DDD
FGG#DDHDD
#GIGDDHDD
FG#GDDHDD
FGGGDDHH#
```

Output Image Test Case 1:

A	A	A	B	B	C	C	Q	D
A	B	B	B	Q	C	E	C	D
A	B	B	B	D	C	Q	C	D
A	Q	A	B	D	C	C	C	D
B	B	B	B	D	Q	D	D	D
F	G	G	Q	D	D	H	D	D
Q	G	I	G	D	D	H	D	D
F	G	Q	G	D	D	H	D	D
F	G	G	G	D	D	H	H	Q

2.3.2 Test Case 2 (Kondisi Normal)

Input Test Case 2::

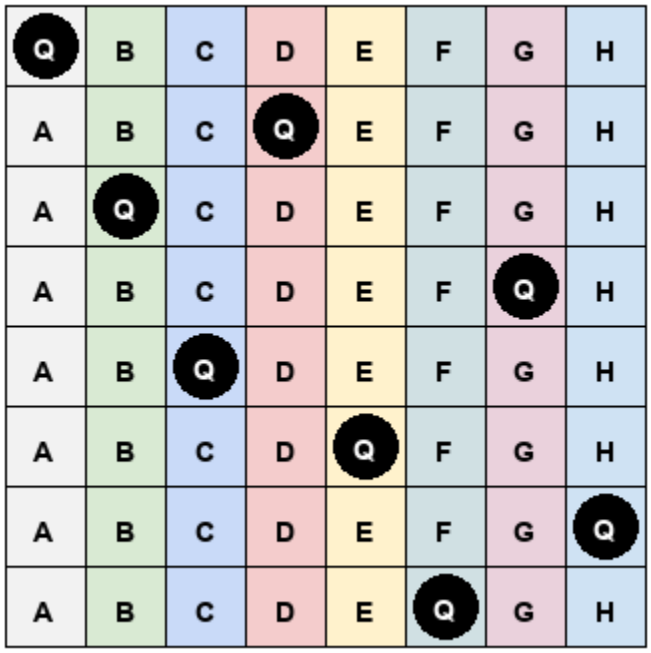
```
AAABBCCCD
ABBBBCECD
ABBBDCECD
AAABDCCCD
BBBBDDDDD
FGGGDDHDD
FGIGDDHDD
FGIGDDHDD
```

```
FGGGDDHHH
```

Output txt Test Case 2:

```
Metode: exhaustive
Waktu pencarian: 2642.7753 ms
#BCDEFGH
ABC#EFGH
A#CDEFGH
ABCDEF#H
AB#DEFGH
ABCD#FGH
ABCDEFG#
ABCDE#GH
```

Output Image Test Case 2:

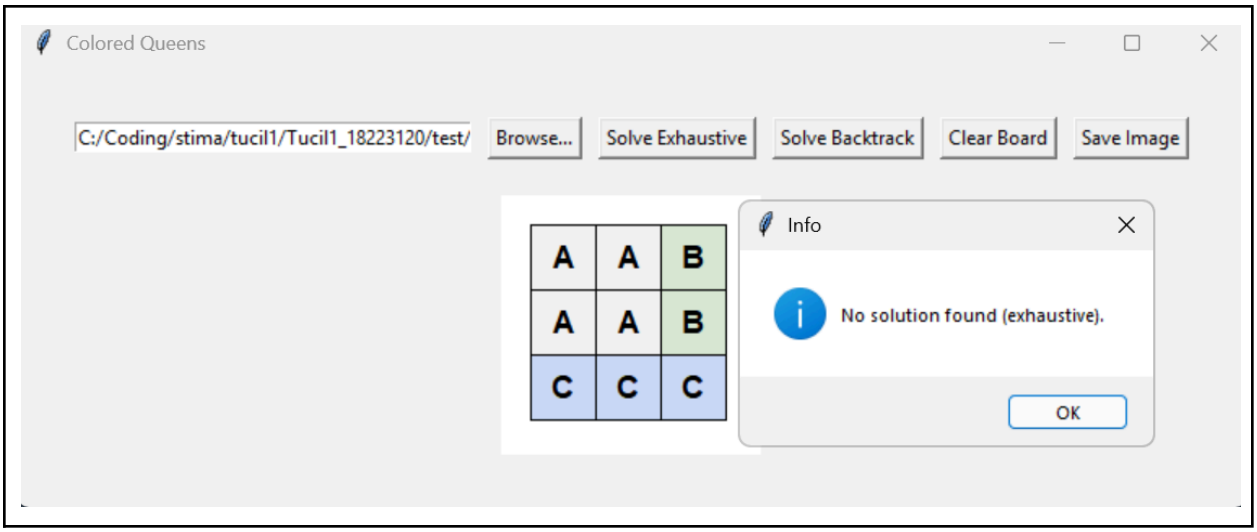


2.3.3 Test Case 3 (No Solution)

Input Test Case 3::

```
AAB
AAB
CCC
```

Output Test Case 3:



2.3.4 Test Case 4 (Kondisi Normal)

Input Test Case 4::

```
AAABBCCCB
DABBBBCBB
DAEEEBCBB
FFFEBBBBB
HFHEHHHII
HFHHHHHIH
HHHJJJHIH
HHHHJHHHH
HHHHJHHHH
```

Output txt Test Case 4:

```
Metode: exhaustive
Waktu pencarian: 130205.2071 ms
AA#BBCCCB
DABBBB#BB
#AEEEBCBB
FFFEB#BBB
HFH#HHIII
H#HHHHHIH
HHHJJJH#H
HHHH#HHHH
HHHHJHHH#
```

Output Image Test Case 4:

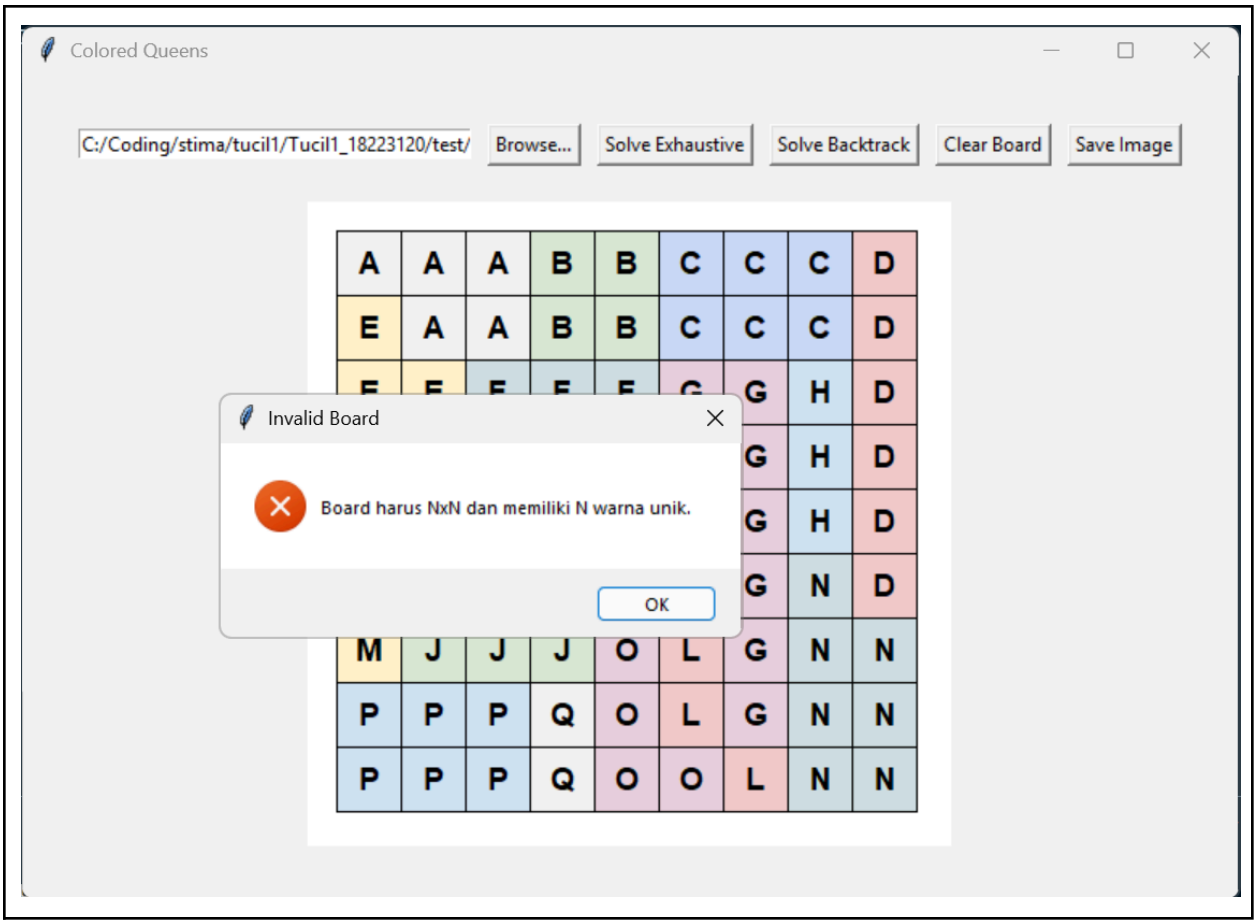
A	A	Q	B	B	C	C	C	B
D	A	B	B	B	B	Q	B	B
Q	A	E	E	E	B	C	B	B
F	F	F	E	B	Q	B	B	B
H	F	H	Q	H	H	I	I	I
H	Q	H	H	H	H	H	I	H
H	H	H	J	J	J	H	Q	H
H	H	H	H	Q	H	H	H	H
H	H	H	H	J	H	H	H	Q

2.3.5 Test Case 5 (Input Invalid)

Input Test Case 5:

```
AAABBCCCD
EAABBCCCD
EEFFFGGHD
IIFFFGGHD
IIJKKLGHD
MMJKKLGND
MJJJOLGNN
PPPQOLGNN
PPPQOOLNN
```

Output Test Case 5:

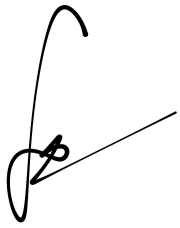


2.4. Link Repository Github

Berikut adalah link repository yang berisi source code dari program algoritma brute force yang diimplementasikan:
https://github.com/LeonArif/Tucil1_18223120

2.5. Surat Pernyataan

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.



Leonard Arif Sutiono

3. Lampiran

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	✓	
2	Program berhasil di jalankan	✓	
3	Solusi yang diberikan program benar dan mematuhi aturan permainan	✓	
4	Program dapat membaca masukan berkas .txt serta menyimpan solusi dalam berkas .txt	✓	
5	Program memiliki Graphical User Interface (GUI)	✓	
6	Program dapat menyimpan solusi dalam bentuk file gambar	✓	